RÉCAPITULATIF DE SESSION — 2 SEPTEMBRE 2026

AI Terraform Landing Zone Generator

Diagnostic complet de l'application, investigation sur les erreurs de génération IA/Terraform, et mise en place d'un pipeline résilient qui ne casse jamais même quand le modèle IA produit du code imparfait.

Projet : AI-Terraform-Landing-Zone-Generator · Stack : FastAPI (backend) + Vue 3 (frontend) + Groq API (IA) + Terraform CLI (validation)

1. Ce qu'est l'application

Une app web qui génère une **AWS Landing Zone Terraform** (VPC, subnets, routing, sécurité, CloudTrail, CloudWatch, logs S3...) à partir d'un formulaire, en passant par un modèle IA hébergé sur **Groq**.

Composant	Rôle	Fichiers clés
Frontend	Formulaire (Vue 3 via CDN) + affichage du résultat	`frontend/index.html`, `app.js`, `style.css`
API	Route `POST /api/generate` et `GET /api/download/{projet}`	`backend/routes/generate.py`
Prompt	Construit le prompt envoyé à l'IA à partir des paramètres du formulaire	`backend/prompts/landing_zone.py`
Appel IA	Appelle Groq (modèle `openai/gpt-oss-120b`), gère les erreurs réseau/API	`backend/services/ai/terraform_ai_service.py`
Parsing	Découpe la réponse texte de l'IA en fichiers `.tf` individuels	`backend/parser/terraform_parser.py`
Sauvegarde + Zip	Écrit les fichiers sur disque, crée l'archive téléchargeable	`backend/services/terraform_service.py`, `backend/utils/zip_manager.py`
Validation	Exécute `terraform init` puis `terraform validate`	`backend/services/terraform_validator_service.py`

Flux complet :

```
Formulaire → construction du prompt → appel Groq → parsing de la réponse
→ écriture des fichiers .tf → terraform init/validate → création du .zip
→ réponse JSON (fichiers, warnings, résultat validation, lien de téléchargement)
```

2. Diagnostic initial

Démarche suivie pour analyser l'application avant de toucher au code :

1. **Lecture complète du code** : tous les fichiers backend (routes, services, parser, prompts, config, exceptions) et frontend, pour comprendre l'architecture réelle avant de juger quoi que ce soit.

2. **Vérification de l'hygiène du dépôt** : `.env` bien ignoré par git (pas de fuite de clé API), `generated/` non versionné.
3. **Lancement de la suite de tests** (`pytest`) : tous verts avant toute modification → confirme qu'il n'y a pas de régression déjà présente.
4. **Test réel bout en bout** : un vrai appel `POST /api/generate` (pas juste une lecture de code) pour observer le comportement effectif face à la vraie API Groq.

> **Résultat du test réel :** le pipeline fonctionnait techniquement (réponse HTTP 200), mais le Terraform généré par l'IA était **invalide** : `terraform validate` échouait.

Bugs identifiés

BUG `aws_eip.nat` contenait *à la fois* `vpc = true` (argument supprimé du provider AWS actuel) et `domain = "vpc"` (le bon argument) — alors que le prompt interdisait explicitement `vpc = true`.

BUG `aws_cloudtrail.trail` utilisait `is_logging = true`, un argument qui n'existe pas sur cette ressource.

OBSERVATION Le parser supprimait toute l'indentation des fichiers `.tf` générés (`raw_line.strip()` appliqué à chaque ligne) — cosmétique, sans impact sur la validité.

3. Est-ce un problème de l'API Groq ?

Question posée en cours de session : le souci vient-il de l'API Groq elle-même ? Plutôt que de deviner, la question a été testée empiriquement.

Ce qui n'est pas le problème

L'infrastructure Groq répond correctement : pas d'erreur, pas de timeout, réponse complète reçue à chaque appel.

Ce qui est lié à l'usage fait de Groq

> **Découverte :** le compte Groq utilisé est sur le tier `on_demand`, limité à **8000 tokens/minute (TPM)**. Une requête à 8983 tokens s'est fait rejeter (`429 rate_limit_exceeded`). C'est très probablement pour rester sous cette limite que le code utilise `max_tokens: 6000` et `reasoning_effort: "low"`.

Test comparatif : reasoning_effort low vs medium

Le même prompt a été rejoué avec `reasoning_effort: "medium"` (en restant sous la limite de tokens) :

Réglage	Résultat de `terraform validate`
`low` (réglage actuel de l'app)	`vpc = true` + `domain = "vpc"` en double sur `aws_eip`, plus `is_logging` invalide sur `aws_cloudtrail`
`medium`	Ces deux erreurs disparaissent, mais une **nouvelle** apparaît : `name` invalide sur `aws_iam_role_policy_attachment`

> **Conclusion :** ce n'est ni un bug du code applicatif (le prompt, le parser et le validateur font correctement leur travail), ni une panne de l'API Groq. C'est une limite de fiabilité inhérente au modèle choisi (`openai/gpt-oss-120b`, rapide et économique) sur une tâche de génération longue avec des contraintes précises. Le comportement est **intermittent**, pas déterministe : parfois ça passe, parfois non, et l'erreur change d'un appel à l'autre.

4. Décision produit : résilience plutôt que blocage strict

Consigne donnée : l'application n'a pas besoin de garantir un Terraform 100% valide à chaque fois — elle doit simplement **ne jamais casser** et toujours produire un résultat exploitable, même quand l'IA se trompe.

Avant / Après

Situation	Avant	Après
Réponse IA tronquée (`finish_reason == "length"`)	Erreur 502, rien généré	Le contenu partiel est conservé et parsé
Un fichier requis manquant (ex : README.md oublié)	Tout le lot rejeté	Les fichiers obtenus sont sauvegardés ; les manquants sont listés dans un champ `warnings`
Un fichier avec un contenu vide	Tout le lot rejeté	Ce fichier est ignoré, les autres sont conservés
`terraform validate` échoue	Déjà toléré (zip quand même généré)	Inchangé — comportement conservé

> Deux cas restent des échecs francs, car il n'y a alors **rien du tout** à sauvegarder : réponse IA totalement vide, ou aucun marqueur de fichier (`### fichier.tf`) reconnu dans la réponse.

Fichiers modifiés

- `backend/services/ai/terraform_ai_service.py` — ne bloque plus sur une réponse tronquée, retourne le contenu partiel tel quel.
- `backend/parser/terraform_parser.py` — ne bloque plus si des fichiers requis manquent ou si un fichier est vide ; nouvelle méthode `missing_required_files()` pour lister ce qui manque sans bloquer.
- `backend/routes/generate.py` — calcule les fichiers manquants et les expose dans un champ `warnings` de la réponse JSON ; saute `terraform validate` si `provider.tf` ou `main.tf` manquent totalement (validation marquée `"skipped"`) ; garde `status: "success"` dès qu'au moins un fichier a pu être produit.
- `frontend/index.html` + `style.css` — nouveau bandeau ⚠ orange qui affiche les `warnings` quand ils existent.
- `tests/test_terraform_ai_service.py`, `tests/test_parser.py`, `tests/test_generate_route.py` — tests mis à jour pour refléter le nouveau comportement tolérant, plus deux nouveaux tests dédiés à la résilience.

Exemple de réponse JSON avec warning

```
{
  "status": "success",
  "project": "mon-projet",
  "generated_files": ["provider.tf", "main.tf"],
  "warnings": [
    "The AI response was missing some expected files: README.md, outputs.tf, ..."
```

```
  ],
  "validation": { "valid": false, "step": "skipped", ... },
  "download": "/api/download/mon-projet"
}
```

5. Vérifications effectuées

1. **Suite de tests complète :** 82/82 tests verts après les changements (`python -m pytest -q`).
2. **Simulation locale** d'une réponse IA incomplète (fichier vide + fichiers manquants) rejouée à travers le vrai pipeline (parser → sauvegarde → zip), sans appeler l'API Groq → comportement résilient confirmé.
3. **Test réel via l'interface** par l'utilisateur : génération avec le bug `is_logging` sur `aws_cloudtrail` → l'app a quand même répondu avec succès, fichiers générés et zip téléchargeable, erreur affichée clairement dans le bandeau de validation.

6. Commit git

Un commit dédié uniquement aux changements de résilience (8 fichiers) a été créé, avec un message expliquant le *pourquoi* du changement, pas juste le *quoi*. Les changements sans rapport déjà présents dans le dépôt (`backend/prompts/landing_zone.py` , fichiers non suivis `run.sh` et `docs/PERSPECTIVES_ET_NOTIONS.md`) ont été volontairement laissés de côté pour ne pas mélanger les sujets dans un même commit.

> **Point de vigilance :** au moment du commit, le dépôt était en **HEAD détachée** (pas rattaché à une branche). Le commit existe mais n'est pour l'instant relié à aucune branche — à rattacher (ex : `main`) pour ne pas risquer de le perdre lors d'un futur `checkout` .

7. Comment lancer et tester l'application soi-même

1. **Démarrer l'app :**

   ```
   cd AI-Terraform-Landing-Zone-Generator
   ./run.sh
   ```

   Démarre le backend (port 8000) et le frontend (port 5173). `Ctrl+C` pour tout arrêter.
2. **Ouvrir le frontend :** `http://localhost:5173` — remplir le formulaire et cliquer sur « Générer la Landing Zone ».
3. **Lire le résultat :** liste des fichiers générés, bandeau ⚠ warnings (si des fichiers manquent), bloc de validation Terraform (OK ou erreur détaillée avec numéro de ligne), lien de téléchargement du `.zip` .
4. **Tester l'API directement :** Swagger auto-généré sur `http://localhost:8000/docs` — pratique pour voir le JSON brut (y compris le champ `warnings`) sans passer par le frontend.

5. **Revalider que rien n'est cassé côté code :**

```
source .venv/bin/activate
python -m pytest -q
```

Doit afficher les tests en vert.

8. Méthode à réutiliser la prochaine fois

La démarche suivie dans cette session, généralisable à n'importe quel diagnostic futur sur ce projet (ou un autre) :

1. **Lire le code de bout en bout** avant de juger quoi que ce soit — comprendre l'architecture réelle, pas supposer.
2. **Faire tourner les tests unitaires** en premier pour détecter des régressions objectives, indépendamment de toute hypothèse.
3. **Faire un vrai appel bout en bout** (pas seulement lire le code) — le comportement réel d'une IA externe ne se devine pas, il se teste.
4. **Isoler une seule variable à la fois** quand un doute porte sur un service externe : rejouer le même prompt en changeant un seul paramètre (ex : `reasoning_effort`) pour voir si le comportement change vraiment.
5. **Distinguer bug de code vs limite d'un service/modèle externe** — ne pas corriger dans le code un problème qui vient d'ailleurs, et vice versa.
6. **Aligner le niveau de rigueur sur le besoin réel du produit** — ici, résilience et disponibilité du résultat priment sur la garantie d'exactitude à 100 %.
7. **Mettre à jour les tests en même temps que le comportement change** — un changement de comportement sans mise à jour des tests correspondants est un changement non vérifié.
8. **Committer avec un message qui explique le pourquoi**, pas seulement la liste des fichiers touchés.